

Universidad Tecnológica de Bolívar

Maestría en Ingeniería

Informe de Ejercicios Propuestos

Asignatura: Deep Learning

Estudiante: Harold Styven Lagares De Voz

Profesor: Jairo Serrano Castañeda

Posgrado: Maestría en Ingeniería

Institución: Universidad Tecnológica de Bolívar

Fecha: Abril de 2026

Este informe sintetiza los resultados y conclusiones de los ejercicios desarrollados en los 13 notebooks de la asignatura de Deep Learning.

Índice

1. Introducción	3
2. Notebook 01: Introducción al Machine Learning	3
2.1. Selección de características e impacto en la precisión	3
2.2. Validación frente a un clasificador de referencia	3
2.3. Reflexión	3
3. Notebook 02: Preprocesamiento y Visualización	4
3.1. Comparación de métodos de escalado	4
3.2. Optimización integral mediante pipelines	4
3.3. Reflexión	4
4. Notebook 03: Modelos Clásicos de Machine Learning	4
4.1. Comparativa de algoritmos en el dataset Wine	4
4.2. Ensamble por votación	5
4.3. Reflexión	5
5. Notebook 04: Redes Neuronales de Capa Densa (MLP)	5
5.1. Eficiencia del Early Stopping	5
5.2. Impacto del volumen de datos	5
5.3. Reflexión	5
6. Notebook 05: Redes Neuronales Convolucionales (CNN)	6
6.1. CNN base en Fashion MNIST	6
6.2. Arquitectura profunda por bloques (VGG-Style)	6
6.3. Reflexión	6
7. Notebook 06a: RNN/LSTM — Predicción Climática	6
7.1. Efecto de la longitud de la secuencia (ventana temporal)	6
7.2. Modelo multivariado con temperatura mínima y máxima	7
7.3. Generalización geográfica	7
7.4. Reflexión	7
8. Notebook 06b: Comparativa de Arquitecturas Recurrentes	7
8.1. SimpleRNN, LSTM, GRU y LSTM Bidireccional	7
8.2. Reflexión	8
9. Notebook 07: Transformers y Atención	8
9.1. Clasificación Zero-Shot	8
9.2. Generación de texto autorregresiva (GPT-2)	8
9.3. Interpretabilidad mediante mapas de atención	8
9.4. Reflexión	8
10. Notebook 08: Redes Generativas Antagónicas (GANs)	9
10.1. GAN densa en Fashion MNIST	9
10.2. DCGAN con capas convolucionales	9
10.3. Reflexión	9

11.Notebook 09: Autoencoders	9
11.1. Autoencoder convolucional	9
11.2. Autoencoder Variacional Condicional (CVAE)	10
11.3. Reflexión	10
12.Notebook 10: Clustering y Reducción de Dimensionalidad	10
12.1. K-Means vs. DBSCAN en datos de alta dimensionalidad	10
12.2. PCA vs. t-SNE	10
12.3. Reflexión	10
13.Notebook 11: Interpretabilidad de Modelos	11
13.1. SHAP en modelos de árbol: Random Forest vs. Gradient Boosting	11
13.2. SHAP en redes neuronales: KernelExplainer	11
13.3. Reflexión	11
14.Notebook 12: CPU, GPU y Aceleración de Hardware	11
14.1. Efecto del <i>batch size</i> en CPU y GPU	11
14.2. Benchmark con ResNet50 y CIFAR-10	12
14.3. Reflexión	12
15.Notebook 13: Despliegue de Modelos	12
15.1. API REST con FastAPI y validación de entrada	12
15.2. Contenedorización con Docker	12
15.3. Seguimiento de experimentos con MLflow	12
15.4. Reflexión	13
16.Conclusiones Generales	13
Referencias	14

1. Introducción

El presente informe documenta los resultados y conclusiones derivados de la resolución de los ejercicios propuestos en los 13 notebooks de Deep Learning. El recorrido abarca desde los fundamentos del aprendizaje de máquina clásico hasta técnicas avanzadas de despliegue de modelos, pasando por arquitecturas profundas, modelos generativos, transformers y herramientas de interpretabilidad.

El objetivo de este documento no es reproducir el código implementado, sino articular de manera formal las observaciones, comparaciones y aprendizajes que emergieron de cada experimento, ofreciendo una visión integrada y reflexiva de los temas tratados.

2. Notebook 01: Introducción al Machine Learning

2.1 Selección de características e impacto en la precisión

En el primer conjunto de ejercicios se exploró el comportamiento de un modelo de regresión logística aplicado al dataset Iris bajo distintas condiciones experimentales. Al reducir el espacio de entrada de cuatro variables a únicamente dos (longitud y ancho del pétalo), la precisión del modelo disminuyó de 96.67 % a 93.33 %. Este resultado confirma que las dimensiones del pétalo concentran la mayor parte del poder predictivo del conjunto de datos, pero que las variables del sépalos aportan información complementaria que, aunque marginal, es estadísticamente relevante para alcanzar el desempeño máximo.

2.2 Validación frente a un clasificador de referencia

La comparación del modelo entrenado con un *DummyClassifier* de clase más frecuente evidenció la brecha real de aprendizaje: mientras el clasificador de referencia apenas alcanzó el 33.33 % de precisión (correspondiente a una predicción aleatoria uniforme en un problema de tres clases balanceadas), la regresión logística alcanzó el 96.67 %, lo que representa una mejora efectiva de 63.33 puntos porcentuales. Este tipo de comparación es fundamental para validar que un modelo está capturando patrones estadísticos reales y no simplemente reflejando la distribución de clases.

2.3 Reflexión

La regresión logística demostró ser una herramienta de alta eficiencia para problemas con separabilidad lineal. La experiencia resalta que la selección de variables y la comparación sistemática con baselines son pasos esenciales en cualquier flujo de trabajo de aprendizaje automático.

3. Notebook 02: Preprocesamiento y Visualización

3.1 Comparación de métodos de escalado

Se evaluaron tres estrategias de normalización mediante validación cruzada de cinco pliegues. El **StandardScaler** obtuvo la mayor precisión media (0.9807) con una desviación estándar competitiva (0.0065). El **MinMaxScaler** presentó la menor variabilidad, pero una precisión media inferior (0.9613), mientras que el **RobustScaler** fue cercano al primero, aunque ligeramente más inestable (desviación de 0.0089). El resultado sugiere que las variables del conjunto presentan una distribución que se beneficia de ser centrada en cero con varianza unitaria, lo que favorece al **StandardScaler** en la estimación de los límites de decisión logística.

3.2 Optimización integral mediante pipelines

La integración de **SelectKBest**, **StandardScaler** y **LogisticRegression** en un pipeline optimizado con **GridSearchCV** permitió identificar que el conjunto óptimo de variables a retener fue de 28 (de 30 disponibles), con un parámetro de regularización $C = 1$. La precisión cruzada resultante fue idéntica a la del mejor escalador individual (0.9807), lo que indica que las dos variables descartadas introducían ruido o redundancia. Este ejercicio demostró el valor metodológico de automatizar la búsqueda de hiperparámetros para garantizar decisiones objetivas y reproducibles.

3.3 Reflexión

El uso de pipelines combinados con validación cruzada representa el estándar de la industria para la toma de decisiones basada en evidencia estadística, eliminando el sesgo que puede introducir una única partición de datos.

4. Notebook 03: Modelos Clásicos de Machine Learning

4.1 Comparativa de algoritmos en el dataset Wine

Al aplicar cuatro algoritmos clásicos (Regresión Logística, KNN con $K = 5$, Árbol de Decisión y SVM con kernel RBF) al dataset Wine, se observó un empate en el primer lugar entre la Regresión Logística, KNN y SVM, todos con una precisión del 97.22 %. El Árbol de Decisión quedó ligeramente rezagado con un 94.44 %. El hecho de que un modelo lineal iguale a uno tan complejo como SVM indica que, una vez que los datos son correctamente escalados, las fronteras de decisión del dataset Wine son relativamente claras.

4.2 Ensamble por votación

El clasificador de votación (*soft voting*) combinando los cuatro modelos alcanzó exactamente el mismo nivel de precisión que los mejores individuales (97.22 %), sin ninguna mejora. Este resultado ilustra una limitación fundamental de los métodos de ensamble: su eficacia depende de que los modelos constituyentes cometan errores en instancias distintas (diversidad de errores). Cuando los modelos coinciden en sus aciertos y errores, el ensamble simplemente refuerza la mayoría ya dominante sin encontrar casos adicionales que corregir.

4.3 Reflexión

Para problemas de baja dimensionalidad con fronteras bien definidas, los modelos clásicos continúan siendo altamente competitivos. El escalado correcto de los datos y la validación cruzada resultan más determinantes que la complejidad del algoritmo elegido.

5. Notebook 04: Redes Neuronales de Capa Densa (MLP)

5.1 Eficiencia del Early Stopping

Al configurar un modelo de red neuronal densa con un criterio de parada anticipada (*patience* = 5 sobre la pérdida de validación), un entrenamiento originalmente fijado en 100 épocas se detuvo en la época 17, restaurando los pesos de la época 12 como óptimos. La precisión final en el conjunto de prueba fue de 98.61 %. Este resultado demuestra empíricamente el concepto de sobreajuste: más allá del punto óptimo, la red comienza a memorizar el conjunto de entrenamiento, degradando su capacidad de generalización. El *EarlyStopping* no solo protege al modelo de este deterioro, sino que reduce el costo computacional al evitar iteraciones innecesarias.

5.2 Impacto del volumen de datos

Al migrar del dataset reducido de *digits* (sklearn) al MNIST nativo de Keras (60,000 imágenes de 28×28 píxeles), el tiempo de entrenamiento aumentó a 42.71 segundos, pero la red alcanzó una precisión de 97.82 % sobre 10,000 imágenes de prueba en apenas nueve épocas. La rápida caída de la función de pérdida durante las primeras épocas confirma que una mayor cantidad de datos representativos potencia significativamente la capacidad de abstracción de la arquitectura densa.

5.3 Reflexión

El aprendizaje profundo moderno depende tanto de la arquitectura como de la cantidad y calidad de los datos. La combinación de monitoreo inteligente con *callbacks* y un volumen suficiente de ejemplos permite alcanzar sistemas precisos de manera computacionalmente responsable.

6. Notebook 05: Redes Neuronales Convolucionales (CNN)

6.1 CNN base en Fashion MNIST

Una arquitectura convolucional simple (una sola capa `Conv2D`, `MaxPooling2D`, una capa densa y `Dropout`) alcanzó el 90.69 % de precisión en el conjunto de prueba de Fashion MNIST. Este resultado confirma que las prendas de vestir representan un desafío intrínsecamente mayor que los dígitos numéricos, dado que las diferencias entre categorías pueden ser sutiles (por ejemplo, camisa versus camiseta, o zapatilla versus zapato).

6.2 Arquitectura profunda por bloques (VGG-Style)

Un modelo de tres bloques convolucionales con `BatchNormalization` y `MaxPooling2D` progresivo alcanzó apenas el 90.74 % de precisión, una mejora de solo 0.05 puntos respecto al modelo base. Sin embargo, el tiempo por época se triplicó (de aproximadamente 40 a 135 segundos), y el modelo exhibió sobreajuste prematuro, deteniendo su entrenamiento en la época 9 (de 15 máximas). Este caso ilustra el principio de rendimientos decrecientes en Deep Learning: la complejidad adicional no se traduce en mejoras cuando los datos de entrada (imágenes de 28×28 en escala de grises) no poseen la resolución suficiente para justificar una arquitectura de ese nivel de profundidad.

6.3 Reflexión

La complejidad de la red debe ser proporcional a la riqueza informacional de los datos. Arquitecturas como VGG, diseñadas para imágenes de alta resolución en color, no pueden demostrar todo su potencial en imágenes pequeñas de escala de grises.

7. Notebook 06a: RNN/LSTM — Predicción Climática

7.1 Efecto de la longitud de la secuencia (ventana temporal)

Se compararon dos modelos LSTM con ventanas de observación de 7 y 60 días respectivamente para predecir la temperatura promedio. El modelo con ventana corta (7 días) obtuvo un RMSE de 0.7529 °C, mientras que el de ventana larga (60 días) alcanzó solo 0.8000 °C. Contraintuitivamente, una ventana más amplia introdujo información estacional irrelevante que diluyó la señal del pasado inmediato, que es el predictor más eficaz para el clima a corto plazo.

7.2 Modelo multivariado con temperatura mínima y máxima

La incorporación de $tmin$ y $tmax$ como variables adicionales resultó en un RMSE de 0.7567 °C, prácticamente idéntico al modelo univariado. En la zona geográfica analizada (clima tropical de Cartagena), la temperatura promedio diaria presenta alta correlación con sus extremos, lo que genera colinealidad que neutraliza el potencial beneficio de las variables adicionales.

7.3 Generalización geográfica

Al aplicar la metodología entrenada sobre datos de Cartagena a la ciudad de Londres, el RMSE se disparó a 2.9349 °C. Este resultado subraya que los modelos climáticos son hiper-locales: la alta volatilidad estacional, los frentes fríos abruptos y la mayor varianza diaria del clima londinense exigen arquitecturas más profundas, ventanas temporales ajustadas y variables meteorológicas adicionales (humedad, presión, índices estacionales).

7.4 Reflexión

Los patrones aprendidos en un contexto geográfico o climático raramente son transferibles sin adaptación. Comprender la relevancia estadística del pasado disponible es tan importante como la arquitectura del modelo.

8. Notebook 06b: Comparativa de Arquitecturas Recurrentes

8.1 SimpleRNN, LSTM, GRU y LSTM Bidireccional

Se entrenaron cuatro arquitecturas recurrentes durante 20 épocas fijas sobre una serie temporal sintética (onda senoidal). Los resultados se resumen en la Tabla 1.

Cuadro 1: Comparativa de arquitecturas recurrentes sobre onda senoidal.

Arquitectura	Tiempo (s)	MSE en Test
SimpleRNN	11.71	0.001201
LSTM	19.57	0.001037
GRU	19.45	0.001210
Bidirectional LSTM	20.12	0.001113

La LSTM alcanzó el menor error, demostrando que sus tres compuertas internas (olvido, entrada y salida) son las más adecuadas para modelar una señal periódica regular. La GRU, pese a ser más eficiente en parámetros, resultó ligeramente menos expresiva en este caso particular. La red Bidireccional no superó a la LSTM estándar, ya que en señales periódicas puras no existe información contextual adicional al leer la secuencia en sentido inverso.

8.2 Reflexión

El número de parámetros no garantiza mejores resultados. Para señales periódicas claras y sin ruido, una LSTM estándar alcanza el límite de aprendizaje de manera eficiente, haciendo redundantes enfoques más complejos diseñados para contextos más caóticos.

9. Notebook 07: Transformers y Atención

9.1 Clasificación Zero-Shot

Mediante el pipeline `zero-shot-classification` de HuggingFace (modelo *facebook/bart-large-mnli*), se clasificó una oración sobre el automóvil eléctrico de Tesla en categorías personalizadas sin ningún entrenamiento previo. El modelo asignó una probabilidad combinada del 98.9 % entre “Technology” y “Automotive”, con el resto de categorías obteniendo valores marginales. Este resultado valida que los transformers preentrenados a gran escala capturan conocimiento semántico profundo del mundo, utilizable directamente (*out-of-the-box*) sin adaptación al dominio específico.

9.2 Generación de texto autorregresiva (GPT-2)

A partir de un mismo *prompt* determinista, GPT-2 generó tres continuaciones semánticamente distintas aunque gramaticalmente coherentes y temáticamente consistentes. Este comportamiento refleja la naturaleza probabilística de los modelos decodificadores: en cada paso, el modelo calcula la distribución de la siguiente palabra más probable dado el contexto acumulado, lo que abre un espacio de divergencia creativa a partir de un mismo punto de partida.

9.3 Interpretabilidad mediante mapas de atención

Al visualizar los pesos de atención de BERT en capas tempranas y profundas, se confirmó un patrón ampliamente documentado en la literatura: las capas iniciales distribuyen la atención de forma difusa y local (hacia palabras vecinas), mientras que las capas finales concentran la mayor parte de la atención en tokens de puntuación (como el punto o el token [SEP]), utilizándolos como “repositorios” de atención residual para liberar el peso semántico significativo hacia las relaciones de larga distancia que definen el significado global de la oración.

9.4 Reflexión

Los modelos transformer modernos representan un cambio de paradigma: su conocimiento preentrenado permite resolver tareas de clasificación, generación e interpretación de texto con niveles de sofisticación difícilmente alcanzables mediante enfoques supervisados clásicos.

10. Notebook 08: Redes Generativas Antagónicas (GANs)

10.1 GAN densa en Fashion MNIST

La arquitectura MLP (Perceptrón Multicapa) utilizada como generador logró producir siluetas reconocibles de prendas de ropa, pero con notable borrosidad visual. Al aplanar la imagen a un vector de 784 píxeles en la primera capa, la red pierde el contexto bidimensional de los píxeles (relaciones espaciales entre bordes y gradientes), lo que impide que los detalles finos de las texturas sean reproducidos con precisión.

10.2 DCGAN con capas convolucionales

La transición a una arquitectura DCGAN (*Deep Convolutional GAN*) representó la mejora cualitativa más significativa. Las capas `Conv2DTranspose` en el generador y `Conv2D` con *strides* en el discriminador extrajeron características espaciales de manera jerárquica, produciendo trazos más definidos y continuos. El uso de hiperparámetros específicos para estabilizar el entrenamiento adversarial (LeakyReLU con pendiente 0.2, `BatchNormalization`, Adam con *learning rate* de 0.0002 y $\beta_1 = 0,5$) permitió mantener el equilibrio competitivo entre generador y discriminador a lo largo del ciclo de entrenamiento.

10.3 Reflexión

Las GANs densas son el punto de partida ideal para comprender la teoría minimax del aprendizaje generativo, pero las DCGANs son el estándar indispensable en problemas de visión computacional, sentando las bases para los modelos generativos del estado del arte actual.

11. Notebook 09: Autoencoders

11.1 Autoencoder convolucional

Al reemplazar las capas densas por convoluciones (`Conv2D` y `MaxPooling2D` en el encoder; `Conv2DTranspose` y `UpSampling2D` en el decoder), la calidad de reconstrucción de imágenes del dataset MNIST mejoró notablemente respecto a una arquitectura basada en capas densas. La pérdida de validación converge de manera estable a partir de la cuarta época, alcanzando 0.0725 al final del entrenamiento. Las convoluciones preservan las relaciones espaciales locales de los píxeles (bordes, esquinas y gradientes), evitando la borrosidad típica de la reconstrucción puramente matemática.

11.2 Autoencoder Variacional Condicional (CVAE)

El CVAE incorpora el *One-Hot Encoding* de la clase tanto en el encoder como en el decoder, logrando el “desenredo” de la identidad del dígito y su estilo de escritura. Como resultado, fue posible generar de forma determinista variantes de cualquier dígito específico (0–9) muestreando distintos puntos del espacio latente condicionado. Este control preciso sobre la generación representa la frontera directa hacia sistemas más modernos como los modelos de difusión y los sistemas de generación condicionada por texto.

11.3 Reflexión

La evolución del autoencoder clásico al CVAE marca la transición del aprendizaje representacional hacia la generación condicionada y controlada, donde se exige a la red no solo que “imagine” algo plausible, sino que “imagine” exactamente lo parametrizado.

12. Notebook 10: Clustering y Reducción de Dimensionalidad

12.1 K-Means vs. DBSCAN en datos de alta dimensionalidad

Al aplicar ambos algoritmos al dataset *Digits* (imágenes de dígitos aplanadas a 64 dimensiones), K-Means distribuyó los puntos en los 10 clústeres forzados por definición. DBSCAN, con un parámetro $\varepsilon = 4,5$, encontró 12 clústeres de forma autónoma, pero clasificó el 21.81 % de las muestras como ruido. Este resultado evidencia la *maldición de la dimensionalidad*: en 64 dimensiones, las distancias euclidianas entre puntos tienden a homogeneizarse, dificultando que DBSCAN delimite regiones de alta densidad con precisión, lo que hace necesaria una reducción previa de dimensionalidad para aplicarlo eficazmente.

12.2 PCA vs. t-SNE

PCA completó la reducción en una fracción de segundo, pero su proyección bidimensional produjo una nube mezclada donde múltiples clases de dígitos se superponen. t-SNE requirió 54 segundos de cómputo, pero logró formar “archipiélagos” visuales bien separados para cada clase, respetando las vecindades no lineales del espacio original. t-SNE es el método superior para la exploración visual de estructuras de agrupamiento, mientras que PCA conserva su valor para análisis lineales rápidos o como paso previo a otros algoritmos.

12.3 Reflexión

La técnica de reducción de dimensionalidad debe seleccionarse en función del objetivo: PCA para análisis globales y eficiencia computacional; t-SNE para descubrir estructuras locales y visualizar agrupamientos intrínsecos de los datos.

13. Notebook 11: Interpretabilidad de Modelos

13.1 SHAP en modelos de árbol: Random Forest vs. Gradient Boosting

Al aplicar `shap.TreeExplainer` a ambos modelos entrenados sobre el dataset de cáncer de mama, se observó que las características más influyentes fueron consistentes entre ambas arquitecturas de ensamble (especialmente *worst area*, *worst concave points* y *mean concave points*). Esto sugiere que dichas características son robustamente informativas con independencia del algoritmo utilizado. Las diferencias entre modelos se manifestaron en el orden y la magnitud de los valores SHAP, lo que refleja las distintas estrategias de construcción de árboles de cada enfoque.

13.2 SHAP en redes neuronales: KernelExplainer

La aplicación de `shap.KernelExplainer` a la red neuronal de Keras (con precisión del 98 % en el conjunto de prueba) confirmó que el patrón de importancia de características guarda similitudes con el de los modelos de árbol, destacando variables como *worst area*, *worst perimeter* y *worst radius*. La herramienta demostró ser agnóstica al modelo, aunque con un costo computacional significativamente mayor, lo que justifica el uso de un conjunto de fondo reducido y la explicación de un subconjunto de muestras.

13.3 Reflexión

Integrar técnicas de interpretabilidad como SHAP y LIME es fundamental para validar que los modelos aprenden relaciones lógicas, detectar posibles sesgos en los datos y comunicar las decisiones del modelo a audiencias no técnicas. La interpretabilidad no es un lujo, sino un requisito en aplicaciones críticas.

14. Notebook 12: CPU, GPU y Aceleración de Hardware

14.1 Efecto del *batch size* en CPU y GPU

Al variar el tamaño de lote entre 32, 128 y 512 para modelos MLP y CNN, se observó que la GPU mantiene ventajas consistentes en todos los escenarios. Para el modelo CNN, la reducción de tiempo entre CPU y GPU fue drástica en todos los *batch sizes*: por ejemplo, con *batch size* de 32, la CPU requirió 308 segundos mientras que la GPU necesitó solo 21. En la GPU, un *batch size* mayor (512) resultó en el tiempo de entrenamiento más bajo, dado que maximiza la paralelización de las operaciones matriciales en el hardware especializado.

14.2 Benchmark con ResNet50 y CIFAR-10

El entrenamiento de un modelo ResNet50 con pesos preentrenados de ImageNet sobre el dataset CIFAR-10 expuso la diferencia más pronunciada observada: la GPU completó el proceso en aproximadamente 26 segundos, mientras que la CPU requirió 157 segundos, una razón de aceleración de aproximadamente $6\times$. Esta magnitud de diferencia confirma que para modelos de Deep Learning de gran escala, una GPU no es simplemente una optimización de conveniencia, sino un requisito práctico para la experimentación iterativa en tiempos razonables.

14.3 Reflexión

La elección del hardware de cómputo es una decisión de diseño en proyectos de Deep Learning. La CPU puede ser suficiente para modelos pequeños y prototipos, pero la GPU es indispensable para el entrenamiento eficiente de arquitecturas profundas sobre volúmenes de datos de mediana y gran escala.

15. Notebook 13: Despliegue de Modelos

15.1 API REST con FastAPI y validación de entrada

Se construyó una API de producción sobre FastAPI que expone un modelo de clasificación de Iris preentrenado. Las mejoras implementadas incluyeron la validación estricta de entrada mediante Pydantic (exigiendo exactamente cuatro características numéricas por petición) y la devolución de las probabilidades de predicción en formato de diccionario mapeado por nombre de clase (*setosa*, *versicolor*, *virginica*). Este diseño sigue las buenas prácticas de desarrollo de APIs en entornos de producción.

15.2 Contenedorización con Docker

La generación de los archivos `app.py`, `requirements.txt` y `Dockerfile` permitió encapsular el modelo, sus dependencias y el servidor en un contenedor reproducible e independiente del entorno host. El uso de una imagen base ligera (*python:3.11-slim*) y la exposición del puerto 8000 mediante Uvicorn garantizan portabilidad y escalabilidad horizontal en entornos de producción.

15.3 Seguimiento de experimentos con MLflow

La implementación de MLflow para el seguimiento de experimentos permitió registrar automáticamente los hiperparámetros utilizados, las métricas de evaluación y los artefactos del modelo para distintas configuraciones. Ambas ejecuciones probadas (con 10 y 50 estimadores en un RandomForest) alcanzaron una precisión de 1.0 sobre el conjunto de prueba de Iris, lo que ilustra la utilidad de MLflow para comparar condiciones experimentales de forma estructurada y reproducible, especialmente en proyectos con múltiples iteraciones de entrenamiento.

15.4 Reflexión

El despliegue es la etapa donde el valor del modelo se materializa. Las prácticas de contenedorización y seguimiento de experimentos representan el estándar de la industria para garantizar reproducibilidad, trazabilidad y mantenibilidad de los sistemas de Machine Learning en producción.

16. Conclusiones Generales

El recorrido por los 13 notebooks permitió consolidar una visión integral del ciclo de vida del aprendizaje profundo, desde la exploración inicial de datos hasta el despliegue de modelos en entornos productivos. A continuación se enuncian las lecciones transversales más relevantes:

1. **El preprocesamiento es determinante.** La elección del escalador, el tratamiento de valores faltantes y la selección de variables influyen directamente en el rendimiento final, independientemente de la sofisticación del modelo.
2. **La validación cruzada es el estándar.** Una única partición de datos puede inducir conclusiones sesgadas. La validación cruzada permite tomar decisiones objetivas sobre qué algoritmo, escalador o hiperparámetro elegir.
3. **Más complejidad no siempre es mejor.** Tanto en modelos clásicos (ensambles que no mejoran sobre los individuales) como en redes profundas (VGG sobre imágenes pequeñas), la complejidad excesiva puede perjudicar la generalización o simplemente resultar innecesaria.
4. **Los datos son el activo más valioso.** El salto de un dataset pequeño al MNIST completo, o el uso de secuencias más representativas en modelos recurrentes, evidencia que el volumen y la calidad de los datos tiene un impacto mayor que refinamientos arquitectónicos menores.
5. **La interpretabilidad es parte del diseño.** Herramientas como SHAP permiten no solo auditar las predicciones, sino también detectar sesgos y comunicar decisiones a stakeholders no técnicos, lo que es esencial en aplicaciones con impacto real.
6. **El hardware importa.** La GPU no es un accesorio, sino un componente crítico para la experimentación ágil con arquitecturas profundas. La diferencia de $6\times$ en tiempo de entrenamiento con ResNet50 lo ilustra claramente.
7. **El despliegue cierra el ciclo.** Un modelo que no está en producción no genera valor. La contenedorización con Docker y el seguimiento con MLflow son las herramientas mínimas para garantizar que el trabajo experimental sea reproducible y transferible a entornos reales.

Referencias

- Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.
- Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., et al. (2014). Generative adversarial nets. *Advances in Neural Information Processing Systems*, 27.
- Kingma, D. P., & Welling, M. (2014). Auto-Encoding Variational Bayes. *International Conference on Learning Representations (ICLR)*.
- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.
- Documentación oficial de scikit-learn: <https://scikit-learn.org/stable/>
- Documentación oficial de Keras/TensorFlow: <https://keras.io/>
- HuggingFace Transformers: <https://huggingface.co/docs/transformers/>
- MLflow Documentation: <https://mlflow.org/docs/latest/>
- FastAPI Documentation: <https://fastapi.tiangolo.com/>